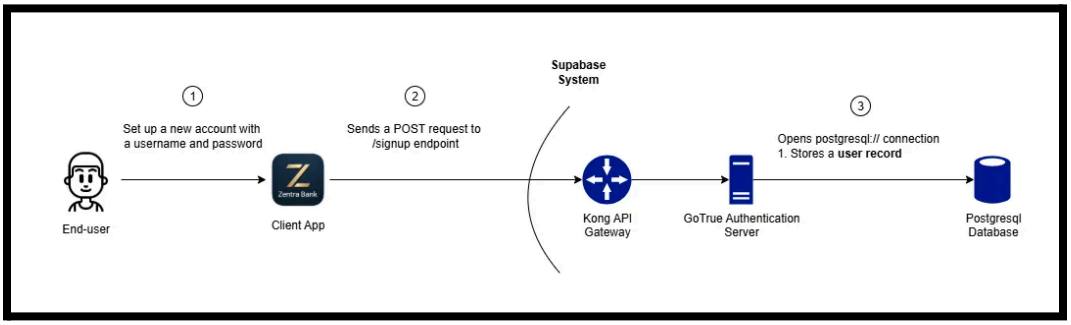
AU2NK for Zentra

Feature ID:
F01. The app allows new registration.

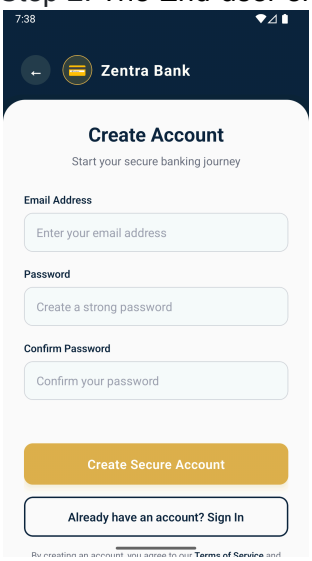
Demonstration:
- Show and explain how one can register.

I am going to show how an end-user can register a new account.

1. The End-user opens the Client App
2. The End-user enters an email address and a password in the sign up form
3. The End-user submits the sign up form to the backend (i.e. Supabase)
4. The End-user receives a response



Step 1: The End-user opens the Client App
Step 2: The End-user enters an email address and a password in the sign up form



Step 3: The End-user submits the sign up form
Behind the scenes, the Client App sends a sign-up request via REST API; /signup to the backend to register a new user.

```
res = await _request(this.fetch, 'POST', `${this.url}/signup`, {
  headers: this.headers,
  redirectTo: options?.emailRedirectTo,
  body: {
    email,
    password,
    data: options?.data ?? {},
    gotrue_meta_security: { captcha_token: options?.captchaToken },
    code_challenge: codeChallenge,
    code_challenge_method: codeChallengeMethod,
  },
```

```
POST /auth/v1/signup HTTP/1.1
authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2Mj
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2Mj
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
x-client-info: supabase-js-react-native/2.49.5-next.1
x-supabase-api-version: 2024-01-01
Content-Type: application/json;charset=UTF-8
Content-Length: 137
Host: vomohccmapvpspaupbah.supabase.co
Connection: keep-alive
Accept-Encoding: gzip, deflate, br
User-Agent: okhttp/4.12.0

{"email":"testtest@gmail.com","password":"123456","data":{},"gotrue_meta_security":{},"code_challenge":null,"code_challenge_method":null}
```

Step 4: The End-user receives a response

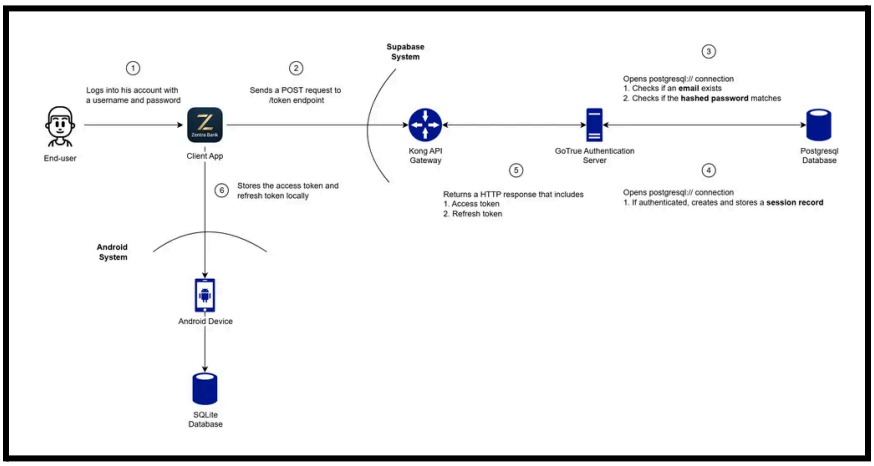
Feature ID:
F02. The app allows sign-in.

Pre-req: F01

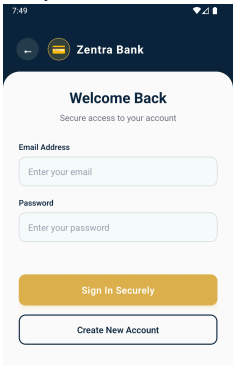
Demonstration:
- Show and explain how one can sign-in.

I am going to show how an end-user can sign-in to their account.

1. The End-user opens the Client App
2. The End-user enters an email address and a password in the sign-in form
3. The End-user submits the sign-in form to the backend (i.e. Supabase)
4. The End-user receives a response



Step 1: The End-user opens the Client App
Step 2: The End-user enters an email address and a password in the sign-in form



Step 3: The End-user submits the sign in form
Behind the scenes, the Client App sends a sign-in request via REST API; /token to the backend to give account access to the user.

```
async signInWithPassword(
  credentials: SignInWithPasswordCredentials
): Promise<AuthTokenResponsePassword> {
  try {
    let res: AuthResponsePassword
    if ('email' in credentials) {
      const { email, password, options } = credentials
      res = await _request(this.fetch, 'POST', `${this.url}/token?grant_type=password`,
        headers: this.headers,
        body: {
          email,
          password,
          gotrue_meta_security: { captcha_token: options?.captchaToken },
        },
```

```
POST /auth/v1/token?grant_type=password HTTP/1.1
authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2N
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2N
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
x-client-info: supabase-js-react-native/2.49.5-next.1
x-supabase-api-version: 2024-01-01
Content-Type: application/json;charset=UTF-8
Content-Length: 72
Host: vomohccmapvpspaupbah.supabase.co
Connection: keep-alive
Accept-Encoding: gzip, deflate, br
User-Agent: okhttp/4.9.2

{"email":"test@gmail.com","password":"123456","gotrue_meta_security":{}}
```

Step 4: The End-user receives a response

A successful response includes a new session - A session is created when a user signs in. By default, it lasts indefinitely and a user can have an unlimited number of active sessions on as many devices. It is represented by:

1. An access token (typically a JWT) which is used by the Client App to access an API
2. A refresh token which is used by the Client App to request for a new access token when the current one expires

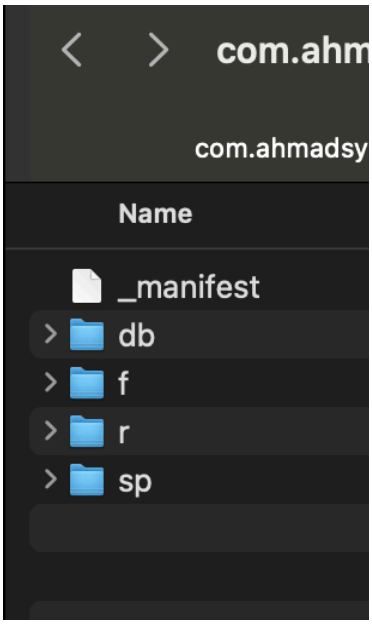
The 2 key components of a session are then stored locally on a SQLite database; RKStorage.

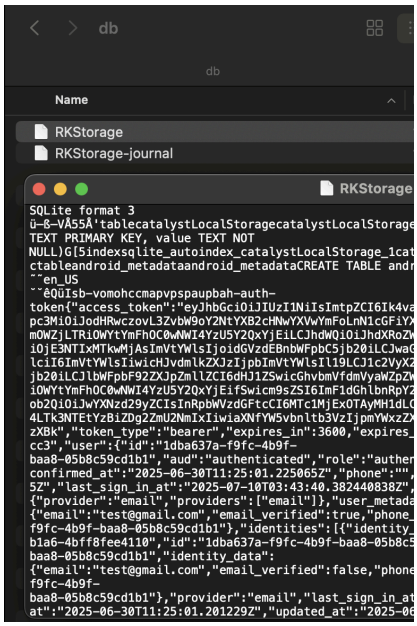
```
HTTP/1.1 200 OK
Date: Thu, 03 Jul 2025 10:30:02 GMT
Content-Type: application/json
Connection: keep-alive
CF-Ray: 9595b4891a9e9d6f-SIN
CF-Cache-Status: DYNAMIC
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Vary: Origin, Accept-Encoding
sb-gateway-version: 1
sb-project-ref: vomohccmapvpspaupbah
X-Content-Type-Options: nosniff
x-envoy-attempt-count: 1
x-envoy-upstream-service-time: 108
Set-Cookie:
__cf_bm=kX_ip_rgYdmuSvXoXqdiYpWWFZKfVaY3CbUflzyp1Kk-1751538602-1.0.1.1-5zVQP19cFcdEdo7f6MbcJYN9.cHIiWjj7DOaMTWSdv2s1pMXXSs3J5iDaMOCN2Odj8.Puh6JuEZmbKeybI1QxGdNKyhsfoZ3xXdH
expires=Thu, 03-Jul-25 11:00:02 GMT; domain=.supabase.co; HttpOnly; Secure
Server: cloudflare
alt-svc: h3=":443"; ma=86400
content-length: 2051

{
  "access_token":
  "eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJzdWIiOiIwNmRm
  Q5Yi05ZWQwYjY1MjYxMTEiLCJhdWQiOiJhdXRoZW50aWNhdGVkIiwiZXhwIjoxNzUxNTQyMjAyLCJpYXQiOjE3NTE1Mzg2MDIsImVtYWlsIjoidGVzdHVzZXJAZXhhbXBsZS5jb20iLCJwaG9uZSI6IiIsImFwcF9tZX
  ciI6ImVtYWlsIiwicHJvdmlkZXJzIjpbImVtYWlsII19LCJ1c2VyX2l1dGFkYXRhIjp7ImVtYWlsIjoidGVzdHVzZXJAZXhhbXBsZS5jb20iLCJlbWFpbF92ZXJpZml1ZCI6dHJ1ZswicGhvbmVfdmVyaWZpZWQiOmZh
  c3Ni1mMjM1LTQxOTktYWQ5Yi05ZWQwYjY1MjYxMTEifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwiOiJhYWwxIiwiYWlyIjpbeyJtZXRob2QiOiJwYXNzd29yZCIsInRpbWVzdGFtcCI6MTc1MTUzODYwMnldLC
  YjkxZGEtYjNkNS00Y2EwLWI0YjgtODM5MjA4ODVhZWNmIiwiaXNfYW5vbnltb3VzIjpmYWxzZX0.vSzFwVyxWv14T_8Yz7KHe5n9WMKABz00AinvhIu6ql4",
  "token_type": "bearer",
  "expires_in": 3600,
  "expires_at": 1751542202,
  "refresh_token": "rghvzadkmdvq",
  "user": {
    "id": "06dfc776-f235-4199-ad9b-9ed0b6526111",
    "aud": "authenticated",
    "role": "authenticated",
    "email": "testuser@example.com",
    "email_confirmed_at": "2025-06-09T03:33:11.43407Z",
    "phone": "",
    "confirmed_at": "2025-06-09T03:33:11.43407Z",
    "last_sign_in_at": "2025-07-03T10:30:02.798301869Z",
    "app_metadata": {
      "provider": "email",
      "providers": [
        "email"
      ]
    },
```

```
    "user_metadata": {
      "email": "testuser@example.com",
      "email_verified": true,
      "phone_verified": false,
      "sub": "06dfc776-f235-4199-ad9b-9ed0b6526111"
    },
    "identities": [
      {
        "identity_id": "f8bca1e1-0902-40b6-989e-923959dfee88",
        "id": "06dfc776-f235-4199-ad9b-9ed0b6526111",
        "user_id": "06dfc776-f235-4199-ad9b-9ed0b6526111",
        "identity_data": {
          "email": "testuser@example.com",
          "email_verified": false,
          "phone_verified": false,
          "sub": "06dfc776-f235-4199-ad9b-9ed0b6526111"
        },
        "provider": "email",
        "last_sign_in_at": "2025-06-09T03:33:11.42987Z",
        "created_at": "2025-06-09T03:33:11.429922Z",
        "updated_at": "2025-06-09T03:33:11.429922Z",
        "email": "testuser@example.com"
      }
    ],
    "created_at": "2025-06-09T03:33:11.424331Z",
    "updated_at": "2025-07-03T10:30:02.805599Z",
    "is_anonymous": false
  }
}
```


RKStorage





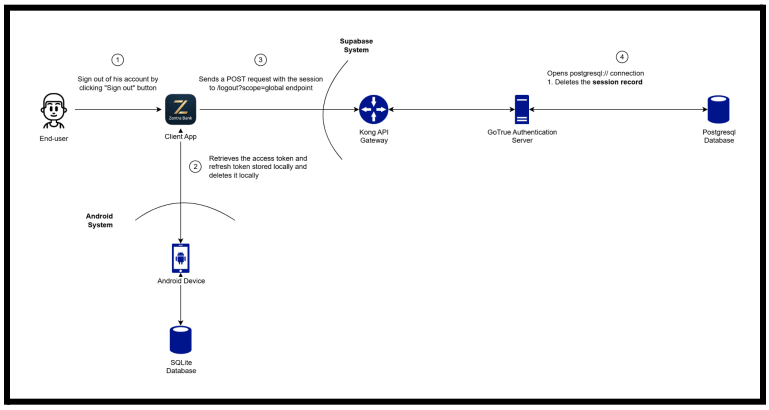
Feature ID:
F03. The app allows sign-out.

Pre-req: F02

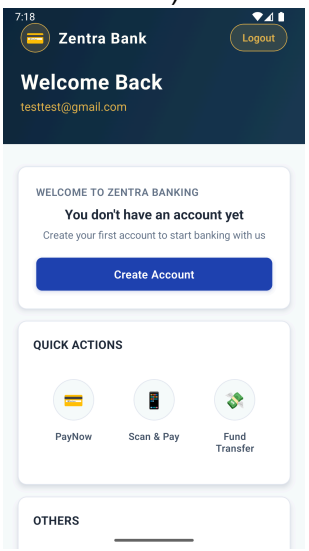
Demonstration:
- Show and explain how one can sign-out.

I am going to show how an end-user can sign-in to their account.

1. The End-user clicks on sign-out button
2. The End-user submits the request to the backend (i.e. Supabase)
3. The End-user receives a response



Step 1: The End-user clicks on sign-out button
Behind the scenes, the Client App retrieves the locally stored session record (JWT and refresh token) from the SQLite database and deletes it.



```
interface Storage {
  /**
   * The **`removeItem()`** method of the Storage interface, when passed a key name, wi
   *
   * [MDN Reference](https://developer.mozilla.org/docs/Web/API/Storage/removeItem)
   */
  removeItem(key: string): void;
  /**
```

Step 2: The End-user submits the request to the backend (i.e. Supabase)
Behind the scenes, the Client App sends a sign-out request via REST API; /logout to the backend to sign the user out of their account.

```
try {
  await _request(this.fetch, 'POST', `${this.url}/logout?scope=${scope}`,
    headers: this.headers,
    jwt,
    noResolveJson: true,
  })
  return { data: null, error: null }
```

Step 3: The End-user receives a response

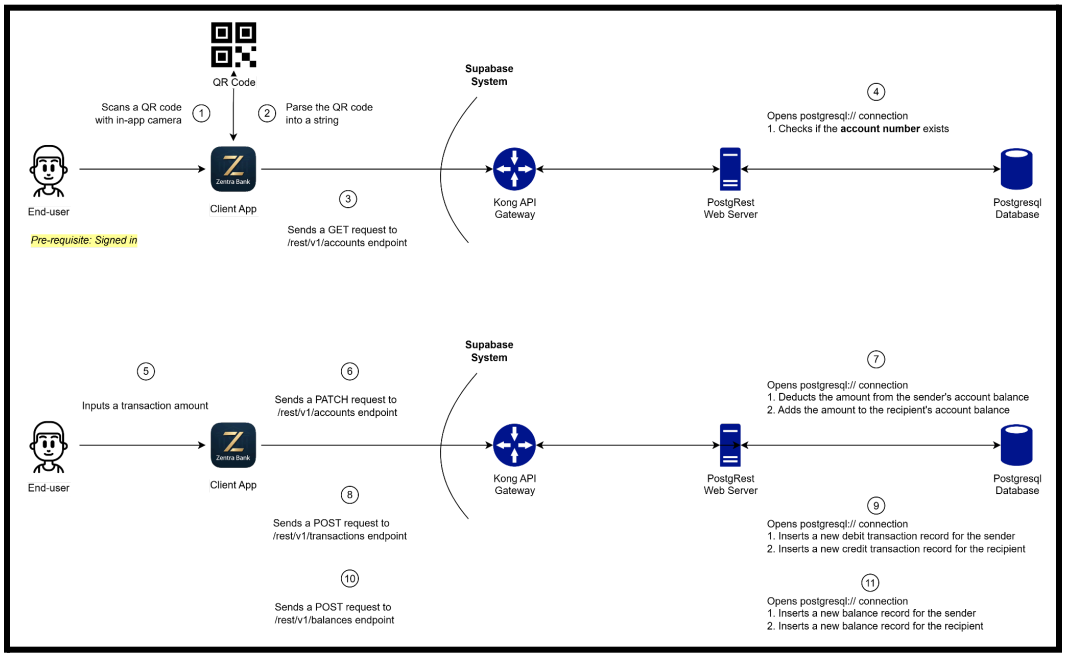
Feature ID:
F04. The app allows fund transfers to other accounts via PayNow

Pre-req: F02

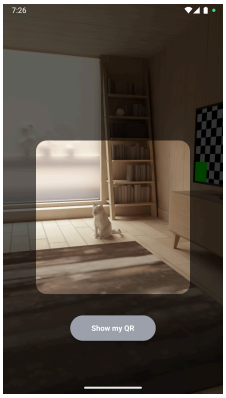
Demonstration:
- Show and explain how an app allows fund transfers to other accounts via PayNow.

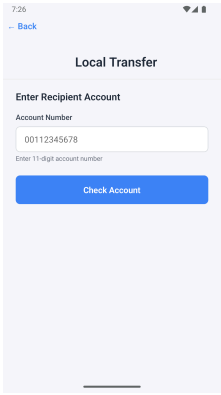
I am going to show how an end-user can transfer funds to other accounts via PayNow.

1. The End-user scans a QR code with the in-app camera and inputs an amount
2. The End-user submits the request to the backend (i.e. Supabase)
3. The End-user receives a response

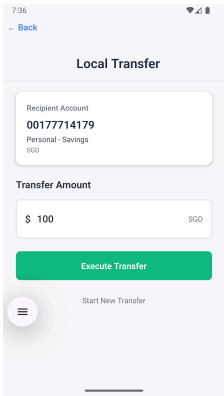


Step 1: The End-user scans a QR code with the in-app camera and inputs an amount
<u>Scans a QR code</u>
Behind the scenes, the Client App parses the QR code into an account number string.





Inputs a number



Step 2: The End-user submits the request to the backend (i.e. Supabase)
Behind the scenes, the Client App 1) sends a request with the account number via REST API; /rest/v1/accounts to the backend to verify if it exists. 2) sends a request with the transfer value via REST API; /rest/v1/accounts, /transactions, /balances to update the change in amount.

Step 3: The End-user receives a response


```
GET /rest/v1/accounts?select=*&account_number=eq.00177714179 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
accept: application/json
accept-profile: public
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc
2F1dGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhd
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZ
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92Z
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0Z
nRpbWVzdGFtcCI6MTclNjg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyL
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
x-client-info: supabase-js-react-native/2.49.5-next.1
```

```
PATCH /rest/v1/accounts?account_id=eq.4fc4ca5e-09d2-433c-9321-add4c91aa6a2 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cH
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKVlQifQ.eyJpc3MiOiJodHRw
2FldGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRo
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJw
l19LCJlc2VyX2lldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImFldGhlbnRpY2F0ZWQiLCJhYWwi
nRpbWVzdGFtcCI6MTclNjg5MjIzMnldLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyLTljNTctMzcl
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 21

{"current_balance":2}
```

Feature ID:
F05. The app allows receipt of push notifications

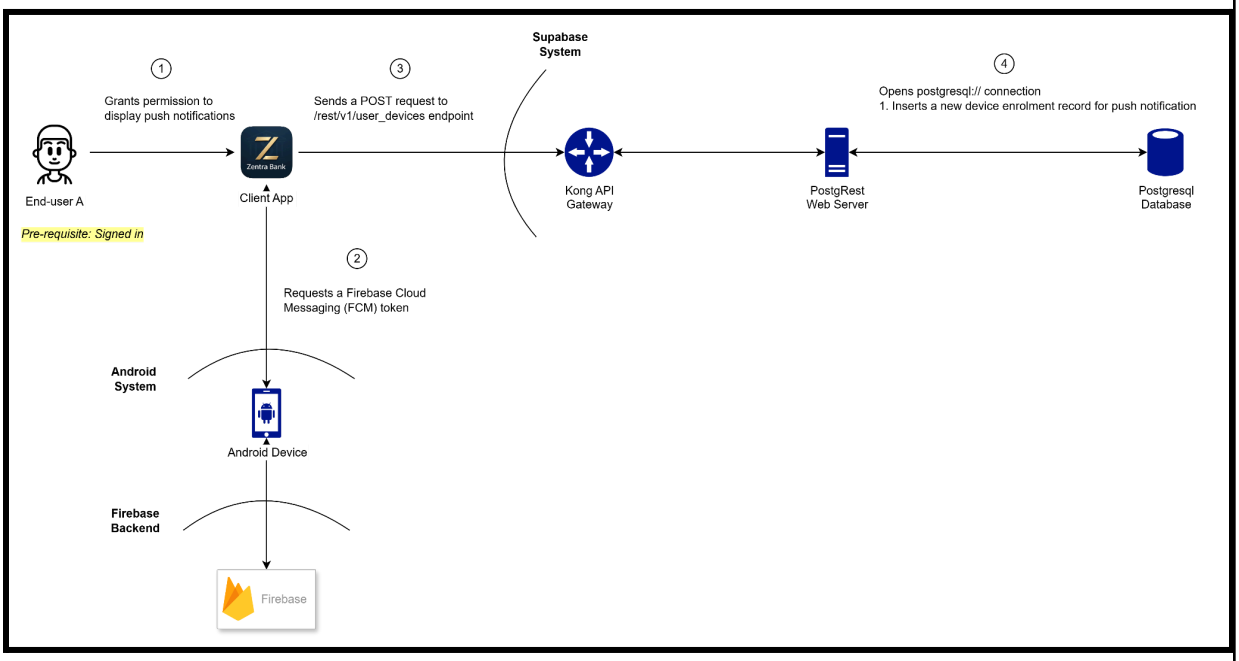
Pre-req: F02

Demonstration:
- Show and explain how an app can receive push notification.

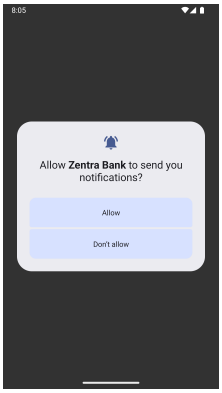
I am going to show how an app receives push notifications

1. The End-user enables push notification
2. The End-user receives push notification of new funds transfer

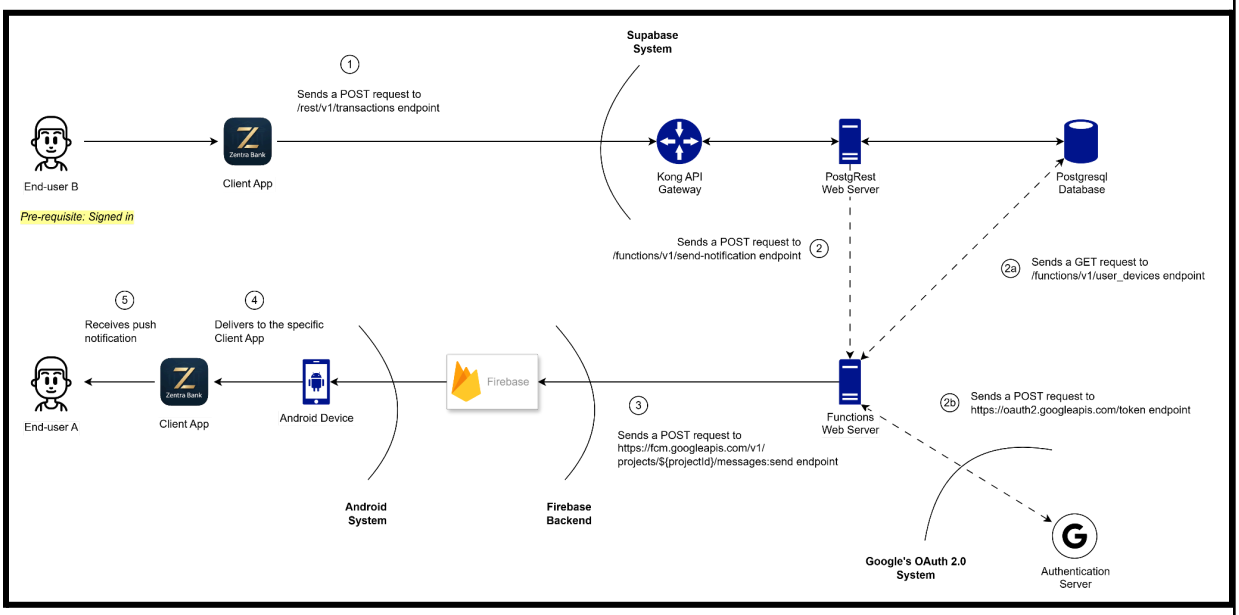
---



Step 1: The End-user enables push notification



Behind the scenes, the End-user grants the Client App permission to display push notifications. Thereafter with the permission granted, the Client App requests a Firebase Cloud Messaging (FCM) registration token via the Firebase SDK, which internally forwards the request to Google Play Services that communicates with the FCM backend - A FCM registration token is a unique string that identifies each client app instance. With the FCM registration token, the Client App makes an insert request via REST API; /rest/v1/user_devices to the Web server.



Step 2: The End-user receives push notification of new funds transfer
Behind the scenes, upon receiving new funds transfer, at the backend-end, the PostgRest Web Server will send a request to inform the new transaction record via REST API; /functions/v1/send-notification to Functions Web Server - a serverless function.

The serverless function will then retrieve the FCM registration token from the Database via Postgres connection and use it to request for an access token via REST API; oauth2.googleapis.com/token to Google's OAuth 2.0 for authorisation.

Finally, the serverless function sends a request consisting of the FCM registration token via REST API; fcm.googleapis.com/v1to FCM backend with the access token to deliver the push notification to the Google Play services of the End-user's device. The Google Play services of the End-user's device then delivers that request to the specific Client App, prompting the push notification.

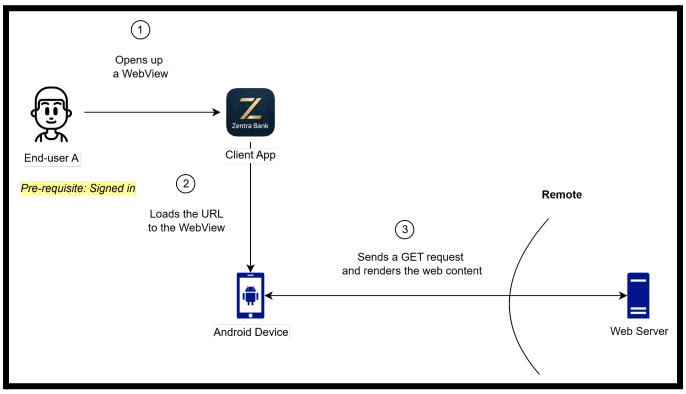
Feature ID:
F06. The app allows a web view.

Pre-req: F02

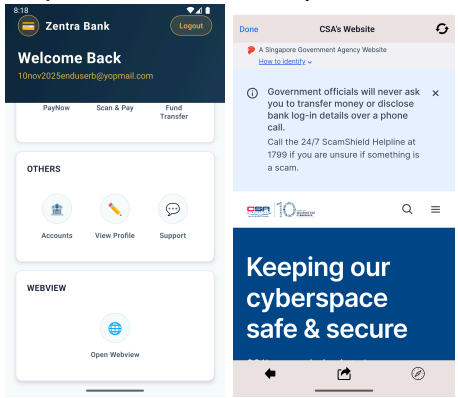
Demonstration:
- Show and explain how an app opens a web view.

I am going to show how an app opens a web view.

1. The End-user opens a web view



Step 1: The End-user opens a web view



Behind the scenes, the Client App loads the URL (i.e. csa.gov.sg) to the Android Operating System WebView, which then sends a GET request and renders the web content as a web view.

Feature ID:
F07. The app allows real-time chat.

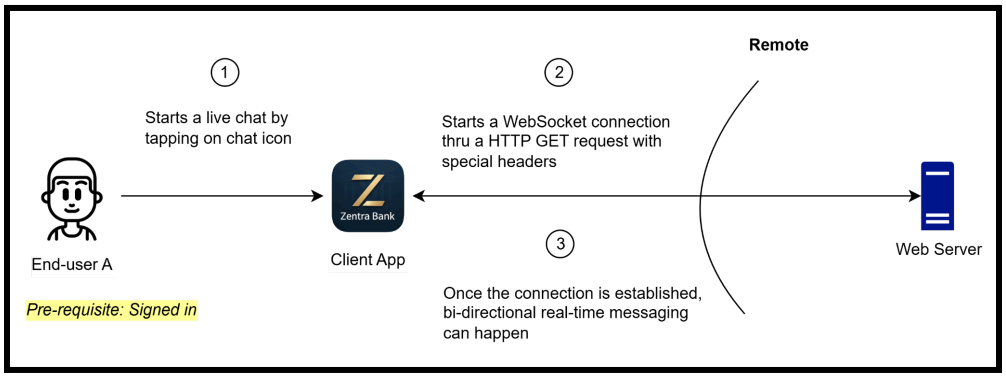
Pre-req: F02

Demonstration:
- Show and explain how an app allows real-time chat.

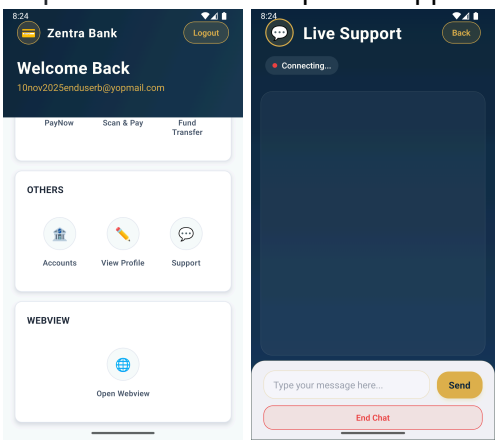
I am going to show how an app allows real-time chat.

1. The End-user opens “support”
2. The End-user starts a real-time connection with a customer support agent

---



Step 1: The End-user opens “support”



Step 2: The End-user starts a real-time connection with a customer support agent
Behind the scenes, the Client App starts a connection to the /ws endpoint of the Web Server using the OkHttp client library. It sends an initial HTTP request with an Upgrade header requesting a WebSocket. The Web Server receives the request, handles the upgrade, and establishes the WebSocket connection.

The WebSocket session is now active, enabling real-time, bi-directional messaging between the end-user and customer support.

Request

Pretty Raw Hex

```
GET /ws?role=customer HTTP/1.1
origin: http://10.0.2.2:8000
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: l+OailxfNgQ8OIeYvkrtfw==
Sec-WebSocket-Version: 13
Host: 10.0.2.2:8000
Accept-Encoding: gzip, deflate, br
User-Agent: okhttp/4.9.2

```